

Использование шаблонного транслятора в системе двоичной трансляции

Н.В.Воронов, Р.А.Савченко

Введение

ЗАО «МЦСТ» осуществляет разработку микропроцессоров серии «Эльбрус» [1] [2] [3] архитектурно несовместимых с платформами (ормо) *# (фирмы, -./01. Поэтому для поддержки большого количества существующих приложений: распространяющихся в виде двоичного кода; возникает задача программной совместимости с *#+1 <а сегоднешней) для одних из &аиболее 3((активных) регистров) &а(о) проблем является программное обеспечение основаемое &а принципами двоичной трансляции [4] работа двоичного транслятора заключается в эквивалентном преобразовании двоичного кода исходной архитектуры в двоичный код целевой. Эквивалентность понимается в том смысле, что в памяти и регистры создаются модели. А контекст А эмулируется архитектурой и эта модель вместе с двоичным транслятором &а(м) кодом; ведет себя в точности как программа &а *#+ процессора. С кодом вводят все *#+ регистры: как общие &а(а) &а(е) так и системные; регистры процессора памяти при трансляции двоичного кода можно проводить оптимизации по следующим (активность результатов кода) &а(тогда) &а(возрастает время трансляции) &а(есть) эту проблему можно применить одновременно несколько уровней трансляции: от быстрого &а(порядка) кода до медленного &а(с &аиболее 3((активных) результатов кодом; и использовать несколько уровней только для &аиболее часто исполняющегося кода [5] [6] [7].

С двоичным оптимизирующим компилятором разрабатываемом в «МЦСТ» есть регистры и интерпретатор: F; B; D; G; 2; и интерпретатор в сущности &е является двоичным транслятором так как получив *#+ команду сразу сам изменяет код &е регистрирует код который будет его моделировать. На блоках транслятор получая &а(в) блоки *#+ кода строит коды следующим образом для каждого *#+ инструкции имеется готовый Эльбрусовский код. А шаблон который с минимальными изменениями прибавляется к трансляции. Оптимизирующие трансляторы сначала переводят *#+ код во внутреннее представление производят оптимизации и затем преобразуют внутреннее представление в результирующий код.

От интерпретации к двоичной трансляции

Для каждой инструкции исходного архитектур' в интерпретаторе есть своя (функция" в'пол'яющаяся &4 ко'текстом такое 5е преобразование :а так5е проявляющаяся все исключитель'е ситуации;" которое 4елает микропроцессор исходного архитектур' во время е@ испол'ения! Чтобы" понять" какую (функцию в'з'вать для текуще) инструкции" е@ с'аала &у5&о 4екодировать! 2олучается" что для каждой инструкции в интерпретаторе в'пол'яются следующие 4е)ствия! 4екодирование" в'зов (функции" преобразование ко'текста" возврат! Кз &и% существе'м является ли6ь преобразование ко'текста! С помощью 4вои8&о) трансляции мо5&о ощутимо уметь6ить время" затрачиваемое &а 4ругие 4е)ствия!

С простым вариантом 4вои8&') транслятор берет" последовательность инструкции и транслирует каждую и% &и% в 4вои8&') ко4 целево) архитектур' :ктор') в'пол'яет 3квивалентное преобразование ко'текста и проявляет исключитель'е ситуации;! 2олучившимся двоично-транслированным кодом мо5&о пользоваться как &ово) (функции)" в'зов которо) 3квиваленте& интерпретации 7рупп' инструкции! 1руппа последователь' % инструкции) &аз'вается базовым блоком" если в &@м &е более 04&о) кома'4' пере%04а" которая" если она есть" стоит после4&е) в блоке! С простым случае 4вои8&') транслятор работает с базов'ми блоками!

Транслируется блок &е б'стрее" чем интерпретируется" &о у5е оттранслирова&&') блок мо5&о со'раить в кеше трансляций и переиспользовать при следующем испол'ении оттранслирова&о) 7рупп' инструкции! Соответствие М*#+ адрес &аала базово) блока" адрес 4вои8&о)транслирова&о) ко4аN при этом &у5&о запом'ить в таблице перекодирования адресов!

С в'беописа&о) с'еме после испол'ения 4вои8&о) транслирова&о) ко4а проис'одит поиск следующей 4вои8&о) транслирова&о) ко4а по таблице перекодирования адресов! Со м'оги% случая% адрес пере%04а" котор'м зака'8ивается базов') блок" является статическим и &е ме'яется в процессе испол'ения программ'! Для устаре&ия изб'то8&' % поисков по таблице" в 4вои8&о) транслирова&ом ко4е сразу 4елается пере%04 &а следующие) 4вои8&о) транслирова&&') ко4! Такая оптимизация &аз'вается связывание блоков [=F!!

Для 4ости5&ия более в'соко) производительности 4вои8&о) транслятора &еоб'одимо введение &есколько% уров'е) трансляции с разли8&о) степе'ю оптимизации ко4а! Для того чтобы" в'4елить &аиболее 8асто работающие) участки ко4а" создается профильный граф! 2ро(иль&') гра(0 3то структура 4а&&' % %ра'ящая и&(ормации о

В числе повторений) ли) и) % участков! <а основами Это) и) (ормации приимается ребение о перетрасляции кода с более в' соким уровнем оптимизации)1

Шаблонная трансляция

Наблюдения) транслятор является втор'м после интерпретатора уровнем систем' и перв'м" &а котором собственно проис%одит 4вои8ая трансляция! Ктерпретатор в процессе своей работ' (ормирует *трассу* А последовательность испол%е% % базов' % блоков! Р074а &екотор') ли) и) % участок в' пол%ился &есколько раз" от &е70 &абирается трасса и запускается 6абло&) транслятор" преобразующий кода4' трасс' в ко4 целевой ар%итектур'! *Входом* в 4вои8&овтра&слирова&ом ко4е &аз'вается адрес" для которого существует запись в таблице переко4ирования адресов! С%одом в трассу является &авало ко54070 базового блока! Мо5&о б'ло б' в'братъ еи&ице) трансляции базов') блок" &о трасса позволяет убрать пере%од' ме54у последователь%ми блоками" со%ра&ив возмо5&ость &авала ко54070 блока б'ть в%одом :как Это б'ло б'" если б' блоки тра&слировались от4ель&о;1 С 4вои8&овтра&слирова&ом ко4е блоки размещаются последователь&о и в пере%ода% &ет &еоб%одимости!1

Ле&ерация ко4а базового блока проис%одит следующим образом: в&авале создается *пролог*" затем ко54ая инструкция тра&слируется от4ель&о" и закар%ивается 4вои8&овтра&слирова&) ко4 *эпilogом*! Если в блоке б'ла инструкция пере%ода" то в соответствии) е) ко4 встраивается 3пилог" и&а&е о& 7е&ерируется от4ель&о! С пролога% и 3пилога% увеличиваются с8@т8ики про(иль&ом70 7ра(а :доступ к с8@т8икам из 4вои8&овтра&слирова&ом70 ко4а осуществляется &е посредстве&о по адресу;" &елается проверка" &е пора ли перетра&слировать ко4 с использова&ием б'стро70 оптимизирующег0 транслятора :&а основами и) (ормации в про(иль&ом7ра(е;1 Так5е" в пролога% проверяется &алибие отло5е& % прер'ва&и)1

Ра54ая инструкция тра&слируется от4ель&о и об'8&о &абирается из &есколько%7отов' % 6аблов :по4отовка опера&4ов" операция" запись результатов;1 Ра54') 6абло& представляет собою 4вои8&') ко4 целевой ар%итектур' с &екотор'ми пуст'ми местами А параметрами 6абло&а! Это могут б'ть &омера ре7истров" ко&стат'" смеще&ия! Для ко540) кода4' в процессе работ' опре4еляется" какие параметр' &у5&о вставить в 6абло&! Среме&&'е ре7истр'" %отя и являются параметрами 6аблов" в рамках% о4&о) инструкции распре4еле&' статически!

За4а&а 4вои8&ом70 транслятора усло5&ается &еоб%одимостью9 коррект&о по4ер5ивать систем&') ко4" в том числе при прер'ва&ия%

в '4авать ко&текст таким 5е" каким о& б'л б' в микропроцессоре ис%о4&о) ар%итектур '1 I ля пов ' 6е&ия произво4итель&ости проявление м&о7и% прер ' ва&и) :о6ибка стра&иц ' " в '%о4 за 7ра&иц ' се7ме&та и тп1; в ' пол&я9тся &a аппарат&ом уров&е" а 4вои8& ') тра&слятор сле4ит только за коррект&ость9 ко&текста1 Ссе в '8исле&ия в 6абло&&ом 4вои8&овтра&слирова&&ом ко4е 4ела9тся &a време&&'% ре7истра%1 Затем произво4ится поп'тка проявить искл98е&ия :&апример" если кома&4а пи6ет в память" то 4вои8&овтра&слирова&&') ко4 пи6ет в память &a 4а&&ом 6а7е;" в том 5е самом поря4ке" в котором и% проявил б' процессор ис%о4&о) ар%итектур ' " и только после 3то7о об&овля9тся 7лобаль&'е ре7истр ' :&апример" , -R.STU.VW- XWV- ./S;1

<а моме&т тра&сляции" 4а5е 4ля пере%о4ов &a ко&ста&т&') а4рес в общем слу8ае ещ@ &еизвест&о" по какому а4ресу бу4ет 4вои8&овтра&слирова&&') ко4" соответству9щи) 4а&&ому пере%о4у1 2о3тому а4реса в 4вои8&овтра&слирова&&'м ко4е" 74е прописа&' смеще&ия пере%о4ов" и соответству9щие им *#+ а4реса со%ра&я9тся в от4ель&о) таблице" записи которо) &аз'ва9т *релокациями*1 Qстестве&&о" 4вои8&овтра&слирова&&') ко4 &ельзя испол&ять" пока релокации &е бу4ут *связаны*1 Связ'ва&ием &аз'вается процесс" в котором по таблице релокации) в 4вои8&овтра&слирова&&') ко4 пропис'ва9тся работа9щие смеще&ия1 I ля релокации" цели котор'% оказались в&утри трасс' " связ'ва&ие проис%о4ит сразу после завер6е&ия тра&сляции1 Остав6иеся релокации со%ра&я9тся вместе с тра&слящие) в ке6е тра&сляции" а связ'ва&ие осуществляется поз5е1 Qсли цели релокации &ет &и в како) тра&сляции" то при соответству9щем пере%о4е бу4ет запусе& и&терпретатор1

С 6абло&&ом ко4е 4ля косве&&'% пере%о4ов :с а4ресом цели" ле5ащем &a ре7истре или в памяти; связ'ва&ие &е 4елається и" в текуще) реализации" пере%о4 все74а осуществляется с использова&ием таблиц' переко4ирова&ия а4ресов1 Сле4ует отметить" 8то в литературе встреча9тся впол&е приме&им'е к 6абло&&ому тра&слятору при&м' оптимизации косве&&'% пере%о4ов [F!" [G!" [Y!" [+!" о4&ако в микропроцессоре «Эльбрус» 4ля 3то7о есть собстве&&') ме%а&изм1 Кспоьзова&ие 4а&&о) возмо5&ости аппаратур' является о4&им из пу&ктов 4аль&е) 6е7о развития 6абло&&о7о тра&слятора1

Набло&&') тра&слятор является ва5&'м зве&ом 4ля обеспе&е&ия сема&тивеско) целост&ости все) систем'1 2ри8и&о) является то" 8то в оптимизиру9щи% компилятора% &екотор'е и&струкции &е реализова&'1 Это обусловле&о сло5&ость9 сема&тики :оптимизации &a4 &е) &е при&осят како7о&либо ощутимо7о 3((екта; или ко74а &е в'4ер5ива9тся оптимисти8&'е пре4поло5е&ия" позволя9щие произвести &еоб&о4им'е оптимизации1 Тра&сляция базов'% блоков" со4ер5ащи% по4об&'е

инструкции" осуществляется только в блоке "м" транслятором

Отладочные технологии

Для отладки и анализа двоичного файла 3 (активировать технологию статического двоичного трансляции! При этом можно обратиться к пользовательскому *#+ коду и обеспечивать трансляцию отдельно! Статически) транслятор" в отличие от "аминеско" работает до запуска файла! Это "акла" вает существование файла и по-прежнему прилагается с самодиагностирующимся кодом "файла" тестируемые коды и запускаемые если происходит перекодировка и преобразования) адрес" транслятора и файлом и возможностью пользоваться! С работой системы статического трансляции для решения всех этих проблем в двоичном файле программа встраивается интерпретатор или "аминеско" транслятор [=G! [=Y! <о" если обратиться к файлу ком-слово "м" файлами" это можно и сделать!

Статическая трансляция позволяет легко получить и посмотреть: в декомпиляторе или отладчике; а двоичный файл) код! С биар) (а) можно так же встроить компилятор и отладчик и (ормации" которую декомпилятор или отладчик понимает и для отображения <а практике очень полезны метки" показывающие" какому *#+ адресу соответствует файл) код и как) (указания) об бл-сегментации!

Производительность

Сравнение производительности разных уровней трансляции! Измерения производились на нескольких замерах из пакетов RZ/USY и RZ/UF>>> [=D! с учетом времени выполнения! Задачи статически транслировались как 54-м уровнем в отладчик! Таким образом" как 54-й) запуск отрабатывает работу результирующей кода только для 04070 уровня трансляции" что позволяет произвести и сравнение! Измерения проводились на потактовом симуляторе микропроцессора «Эльбрус D»! С таблицы = приведено сравнение производительности двоичного файла и кодов для как 54070 уровня" и ормированное относительно 8-го уровня! Что касается интерпретатора" то измерения при "аминеско" трансляции показывающие" что он работает примерно в D> раз медленнее блокового кода!

Кроме времени работы" существуют) характеристики) систем "аминеско" двоичного трансляции является время" затрачиваемое на трансляцию! С таблицы F приведено сравнение времени трансляции разных уровней!

#! h1 xV/`/-nWS`l rk/ eTRRV_-R fS/ pWgV-\1 [TZ/SUWgZT./S j_d/S c0iSTR [//dR.W
|WV- *#+t,fB+G j /0//1 tt j VUSWZSWU/RRWS e/ZWS." ul=D" • 1Fl o/iST_Sa=Y"=\$\$S! O
X_\ /R =B 1
\$! qWSVR q_i_a_-1 cFh r/Uk-W0W\ a _-n ,gZ0/g/-._.VW-1 tt X_S_00/0 XSWU/RRV-\E +.k
,-./S-_.VW-_01 uW0Tg/=\$\$>>|_-T_Sa F>>>1 O X_\ /R =#BF=1
=>1 Qрмолови8 A!C! Метo4' пов'6е&ия произво4итель&ости 4вои8&ов
тра&слиру9щи% систем с аппарат&o) по4ер5ко)! O Иссертация &a
соиска&ие у8е&o) степе&и ка&4и4ата те%&и8ески% &аук" M1" КМСС ?A<"
F>>D!
==1 Солко&ски) C1~1" Лимпельсо& C1I1 Метo4' опре4еле&ия поро7ов
активизации 4и&амивеско7о оптимизиру9ще7о тра&слятора! tt
K&(ормациo&&'е те%&оло7ий" • G" F>> 7
=F! ol q/00_Sn! ,c j y" _`_R. _-n ZWS_i0/ na-_gVU .S_-R0_.WS! tt y[cv,f F>>Y
f--T_0 r/Uk-VU_0 pW-`/S/-U/" оессv,f rS_Ud! F>>Y1 O X_\ /R G=BG+1
=D! [._-n_Sn X/S`WSg_-U/ cb_0T_.VW- pWSZWS_.VW- [Электро&&') ресурс! O ?е5им
4оступаЕ k.ZftmmmlRZ/UjWS/
=G! pSVR.V-_ pV`T/-./R" j Vd/ u_- cgg/SVd" vWSg_- e_gR/a" qSV_- w/mVR! rk/
y-Vb/SRV.a W` ,T//R0_-n qV-_Sa rS_-R0_.WS :y ,qr; oS_g/mWSd! [T-
j VUSWRaR./ gR w_iWS_.WSV/R! F>>=1
=Y1 |VT--B,,/T pk/-" zTT „_-\" r^TB..._- ...T-\" ...W-\B j /- [T" z /VBpkT-\ ...RT! f
[._.VU qV-_Sa rS_-R0_.WS `WS c``VUV/- . j V\S_.VW- W` f e j Bi_R/n fZZ0VU_.VW-R tt .k/
+.k z WsdRkWZ W- sZ.VgV^_.VW-R `WS x[X _-n c g i /nn/n [aR./gR :s x c[;" fZSV0
F>>#